

分布式哈希表

Chord 与 Kademlia 协议

TfOi9

2026.07.24

目录

1 引入

- P2P 思想
- DHT 的基本概念

2 Chord 协议

- 节点拓扑关系: 循环链表
- 加速查找: 手指表
- 动态更新: 稳定化协议
- 容错机制: 后继列表与副本机制

3 Kademlia 协议

- XOR 距离度量
- 节点结构与 k-桶
- 核心算法: 节点查找
- 删除机制

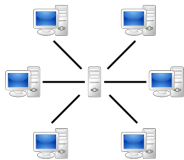
4 应用: BitTorrent

- 技术栈
- 实现内容
- 核心算法
- 当前效果

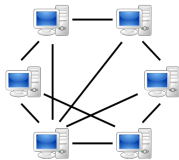
目录

- ① 引入
 - P2P 思想
 - DHT 的基本概念
- ② Chord 协议
- ③ Kademlia 协议
- ④ 应用: BitTorrent

P2P 思想



中心化网络
单点故障 $\Rightarrow$ 服务失效

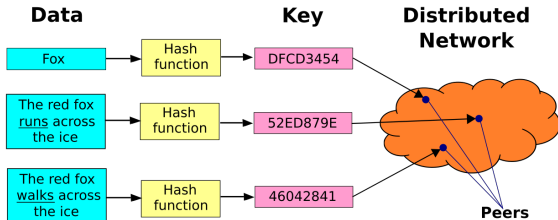


P2P 网络
去中心化 $\Rightarrow$ 高鲁棒性

核心思想

点对点直接通信，无需中心服务器

DHT 基本概念



查找高效

$O(\log N)$ 跳
路由到目标节点

更新高效

单节点变动
仅影响局部

鲁棒性高

少量节点故障
不影响整体

目录

1 引入

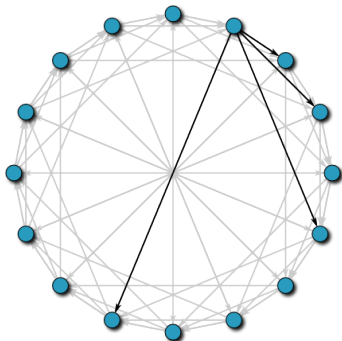
2 Chord 协议

- 节点拓扑关系: 循环链表
- 加速查找: 手指表
- 动态更新: 稳定化协议
- 容错机制: 后继列表与副本机制

3 Kademlia 协议

4 应用: BitTorrent

节点拓扑: 循环链表



哈希值域映射到顺时针环上

- 节点按哈希值在环上排列
- 每个节点维护前驱与后继
- 键值对由直接后继节点负责存储

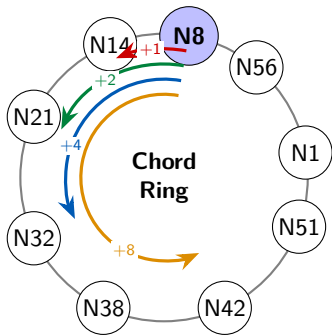
朴素查找的问题

最坏情况需遍历整环: $O(N)$

解决思路

引入手指表 $\rightarrow O(\log N)$ 查找

加速查找: 手指表 (Finger Table)



手指表定义

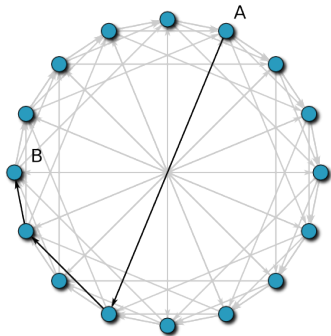
$$\text{finger}[i] = \text{successor}(n.\text{hash} + 2^{i-1})$$

以 N8 为例

+1: N14 +2: N21 +4: N32
 +8: N42

- 手指距离呈指数级增长
- 查找时从大到小遍历
- 复杂度 $O(\log N)$

路由算法: 基于手指表的查找



加速查找的过程

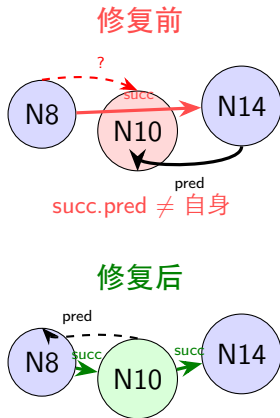
核心算法

```
function n.find_successor(id)                                // 查找某哈希值的直接后继节点
    m = n.find_predecessor(id)
    return m.successor
end function

function n.find_predecessor(id)                              // 查找某哈希值的直接前驱节点
    m = n
    while id ≤ m.hash or id ≥ m.successor.hash do
        m = m.closest_preceding_node(id)
    end while
    return m
end function

function n.closest_preceding_finger(id)                      // 查找节点 n 到 id 的最近手指
    for i = m.downto 1 do
        if n.hash < finger[i].hash < id then
            return finger[i]
        end if
    end for
    return n
end function
```

稳定化协议—原理



核心机制: 定时稳定化循环

每个节点定时运行稳定化操作,
检查并修复可能的错误拓扑。

stabilize() —修复后继

查找后继 S 的前驱 P : 若 P 位于
 N 与 S 之间, 则更新后继为 P 。

notify(m) —更新前驱

若 m 是更好的前驱 (位于
自身之前), 则更新前驱为 m 。

稳定化协议—伪代码

stabilize() 与 notify()

function *n*.stabilize // 周期性检查 *n* 的直接后继,
并通知后继我们的存在

x = *n*.successor.predecessor

if *n*.hash < *x*.hash < *n*.successor.hash **then**

n.successor = *x*

end if

n.successor.notify(*n*)

end function

function *n*.notify(*m*) // *m* 认为它可能是 *n* 的前驱

if *n*.predecessor = nil **or**

n.predecessor.hash < *m*.hash < *n*.hash **then**

n.predecessor = *m*

end if

end function

fix_finger() —刷新手指表

function *n*.fix_finger // 修复 *n* 的手指表

i = random finger index

n.finger[*i*] = find_successor(*n*.hash + 2^i)

end function

说明

- 周期性自动运行
- 各节点独立修复
- 最终收敛到正确拓扑

节点加入与优雅退出

节点加入 (Join)

- 1 新节点 n 通过介绍人 m 入网
- 2 $n.succ = m.find_successor(n.hash)$
- 3 $n.pred = nil$
- 4 稳定化协议自动修复前驱/后继
- 5 向后继请求迁移属于自己的键值对

```
function n.join(m) // 用 m 作为介绍人节点, 让  
新节点 n 加入 DHT 网络  
    n.predecessor = nil  
    n.successor = m.find_successor(n.hash)  
end function
```

优雅退出 (Leave)

- 1 通知前驱更新后继
- 2 通知后继更新前驱
- 3 转移本节点键值对至后继
- 4 稳定化协议自动修复其余指针

容错问题

面临的风险

- 节点可能意外宕机, 未执行优雅退出
- 环形拓扑断裂, 指针悬空
- 宕机节点负责的键值对全部丢失

应对一: 后继列表

- 每个节点维护 r 个后继地址
- 发现后继已死 $\rightarrow$ 取第一个存活节点
- 除非连续 r 个节点同时宕机, 否则环不断裂 (概率极低)

应对二: 副本机制

- 每个节点的键值对复制到 r 个后继
- 查找时若目标节点已死
- 可在后继的副本中检索目标键值
- 删除操作需遍历后继删除副本

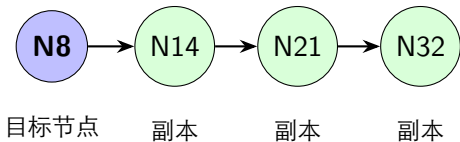
效果

同时应对拓扑断裂 + 数据丢失两层风险

容错机制: 后继列表与副本

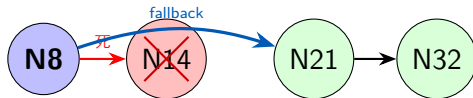
后继列表正常状态

后继列表 ($r = 3$)



节点宕机恢复

节点宕机



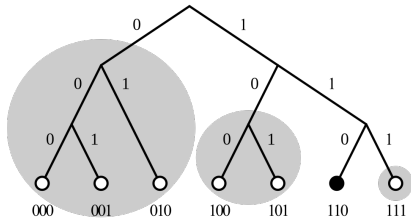
副本机制

各节点键值对复制到 r 个后继中 $\Rightarrow$ 即使 $r-1$ 个节点同时宕机, 数据仍不丢失

目录

- ① 引入
- ② Chord 协议
- ③ Kademlia 协议
 - XOR 距离度量
 - 节点结构与 k-桶
 - 核心算法: 节点查找
 - 删除机制
- ④ 应用: BitTorrent

XOR 距离度量



核心理想

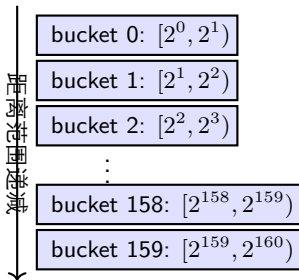
完全忽略节点间显式拓扑关系，
通过 $d(a, b) = a \oplus b$ 度量距离。

XOR 满足距离公理

- ① 非负性: $a \oplus b \geq 0$ ，等号仅当 $a = b$
- ② 对称性: $a \oplus b = b \oplus a$
- ③ 三角不等式: $a \oplus b + b \oplus c \geq a \oplus c$

哈希采用 SHA-1 (160 位); 键值由距离最近节点负责。

节点结构与 k-桶



k-桶结构与 LRU 插入

- 每个节点 160 个 k-桶, 第 i 桶覆盖 $[2^i, 2^{i+1})$
- 每桶容量 K , 按最近访问时间从远到近排列
- 有空间 $\rightarrow$ 尾部插入
- 无空间 $\rightarrow$ Ping 头部 (最久未访问)
已死: 移除并插入新节点
存活: 丢弃新节点, 头部移至尾部

$\Rightarrow$ 防 Sybil 攻击: 旧节点在线时,
路由表不会被恶意节点迅速填充

RPC 方法

四种 RPC 方法

- ① **PING**
连通性检测, 探测节点是否在线
- ② **STORE**
要求某节点存入某键值对
- ③ **FIND_NODE**
返回路由表中距目标最近的 K 个节点
- ④ **FIND_VALUE**
有目标值则返回, 否则同 FIND_NODE

核心机制

- 每次 RPC 调用, 双方尝试把对方插入自己的路由表 (通过 k-桶 LRU)
- FIND_NODE 实现: 从前往后遍历 k-桶, 取距离最小的 K 个即可

意义

节点在正常通信中自然学习
网络拓扑, 无需显式稳定化协议

节点查找算法 (node_lookup)

算法流程

- ❶ 从路由表取距离最近的 K 个节点
→ shortlist
- ❷ 循环, 直到所有节点均被访问:
 - ❶ 从 shortlist 选未访问的最近 α 个
 - ❷ 并行发送 FIND_NODE
 - ❸ 合并返回列表至 shortlist
 - ❹ 保持 shortlist 按距离有序

参数与复杂度

- 并行度 α (通常 $\alpha = 3$)
- 每次迭代, 到目标距离至少折半
- 时间复杂度 $O(\log N)$

基于 node_lookup 的操作

- 插入: node_lookup 找 K 最近节点
→ 并行 STORE
- 查找: 同上, 改调 FIND_VALUE
查到值则提前结束

容错机制与节点加入

容错机制

- 键值对存储在最近的 K 个节点中
- 单一节点宕机不导致数据丢失
- 无显式拓扑—不需修复指针
- k -桶中已死节点逐渐被替代

与 Chord 对比

Chord: 需后继列表 + 稳定化
Kademlia: RPC 通信自然收敛

节点加入 (Bootstrap)

- ① 通过介绍人入网, 加入路由表
- ② 自举: 对自身哈希调 `node_lookup`
- ③ → 填充自身路由表
- ④ → 加入邻居路由表
- ⑤ 从邻居拉取更近的键值对

删除机制

标准机制: 重新发布

- 每一定时间 (如 24 小时) 所有键值对清空
- 键值对发布者根据情况重新发布
- 需删除 → 不重新发布即可
- 简单、无需额外 RPC

强制删除 (非标准)

- ① 查找距离目标键最近的 K 个节点
- ② 向它们发送删除 RPC

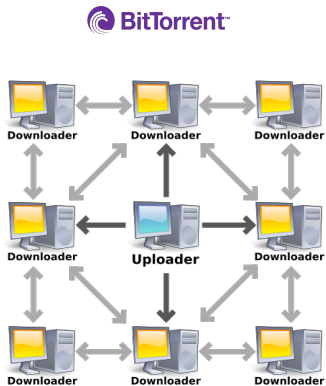
潜在问题

若持有该键值的节点在删除前下线, 删除完成后重新上线, 则残留该键值对。

目录

- ① 引入
- ② Chord 协议
- ③ Kademlia 协议
- ④ 应用: BitTorrent
 - 技术栈
 - 实现内容
 - 核心算法
 - 当前效果

BitTorrent 项目简介



项目架构

组件	语言	说明
DHT 层	Go	Kademlia, 独立进程
后端	Rust	Peer Wire Protocol
通信	gRPC	进程间解耦

基于 Kademlia DHT 的 P2P 文件传输

实现内容

协议与通信

- **Bencode** 编解码—完整解析.torrent 文件
- **Peer Wire Protocol** —握手 + 9 种消息
(Choke/Unchoke/Interested/Have/Bitfield/Request/Piece/Cancel)
- **TCP** 通信—出站连接 / 入站监听
tokio::select! 三路并发事件循环

调度与存储

- **Session** 调度—消息分发、下载管线、SHA1 校验、Have 广播、DHT 发现 Peer
- 存储层—Piece 磁盘读写、SHA1 校验后落盘、单/多文件组装
- **CLI** —download / seed 模式
- **Seed** 模式—加载校验后做种

核心算法

① Kademlia DHT (Go 侧)

基于 XOR 度量, 支持 get_peers 与 announce_peer

② Rarest-First Piece 选择

统计所有 Peer 的 piece 持有频次, 优先下载副本最少的 piece

③ Tit-for-Tat 阻塞算法

每 10s 按上传量选 4 个 unchoke slot; 每 30s 随机乐观 unchoke 一个

④ 5-deep Pipeline

每个 Peer 同时维护最多 5 个 inflight block 请求 (16KB/block)

当前效果与限制

测试结果

- **330+** 单元测试全通过
- 双机跨机器测试: 做种 → 下载, SHA256 一致
- 三机跨机器测试: A 做种, B/C 下载;
B/C 做种后 A 可并行下载

已知限制

- 暂不支持 Tracker 发现
- 无断点续传
- 无前端界面
- 仅支持局域网传输

- ① I. Stoica et al., “Chord: A scalable peer-to-peer lookup service for internet applications,” *ACM SIGCOMM CCR*, vol. 31, no. 4, pp. 149–160, 2001.
- ② P. Maymounkov and D. Mazières, “Kademlia: A Peer-to-Peer Information System Based on the XOR Metric,” in *IPTPS 2002*, LNCS 2429, Springer, 2002, pp. 53–65.
- ③ Wikipedia, “Distributed hash table,” 2026. [Online]. Available: https://en.wikipedia.org/wiki/Distributed_hash_table (accessed Jul. 23, 2026).
- ④ Wikipedia, “Chord (peer-to-peer),” 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Chord_\(peer-to-peer\)](https://en.wikipedia.org/wiki/Chord_(peer-to-peer)) (accessed Jul. 23, 2026).
- ⑤ Wikipedia, “Kademlia,” 2026. [Online]. Available: <https://en.wikipedia.org/wiki/Kademlia> (accessed Jul. 23, 2026).
- ⑥ Wikipedia, “BitTorrent,” Wikipedia, The Free Encyclopedia, 2026. [Online]. Available: <https://en.wikipedia.org/wiki/BitTorrent> (accessed Jul. 24, 2026).
- ⑦ T0morrowarc1, “DHT-ACMPPCA25,” GitHub repository. [Online]. Available: <https://github.com/T0morrowarc1/DHT-ACMPPCA25>
- ⑧ Tf0i9, “DHT,” GitHub repository. [Online]. Available: <https://github.com/Tf0i9/DHT>
- ⑨ Tf0i9, “BitTorrent,” GitHub repository. [Online]. Available: <https://github.com/Tf0i9/BitTorrent>

谢谢大家！

Q & A

<https://github.com/T0morrowarc1/DHT-ACMPPCA25>

<https://github.com/Tf0i9/DHT>

<https://github.com/Tf0i9/BitTorrent>